



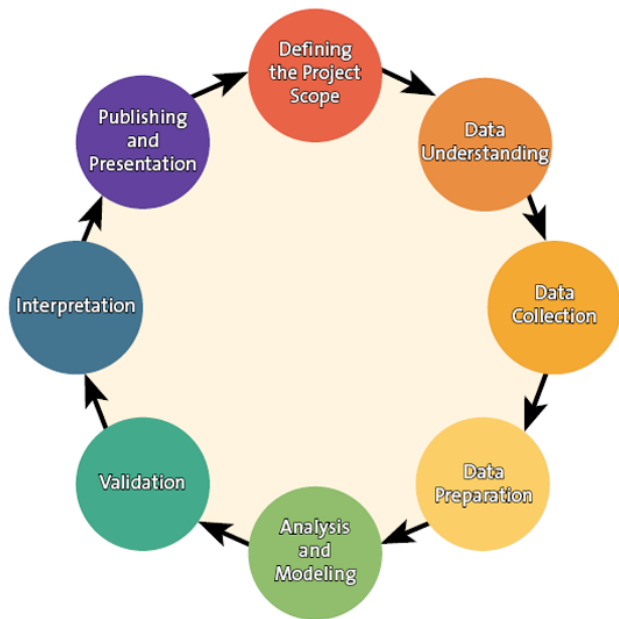
UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

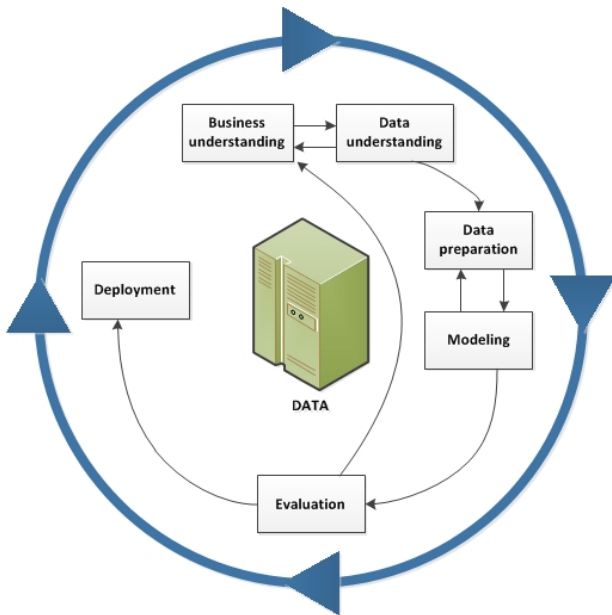
Deployment

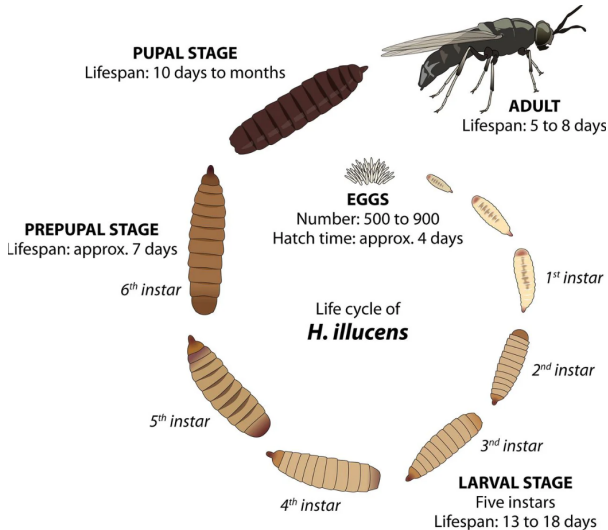
EIN087B - Ciencia de Datos
Paralelo 701

Profesor: Jorge Portilla G.
`jorge.portilla@usm.cl`

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile







"Many insects have a larval form that is optimized for extracting energy and nutrients from the environment and a completely different adult form that is optimized for the very different requirements of traveling and reproduction."¹

¹G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv preprint arXiv:1503.02531, 2015.

Pipeline completo de un sistema de medición

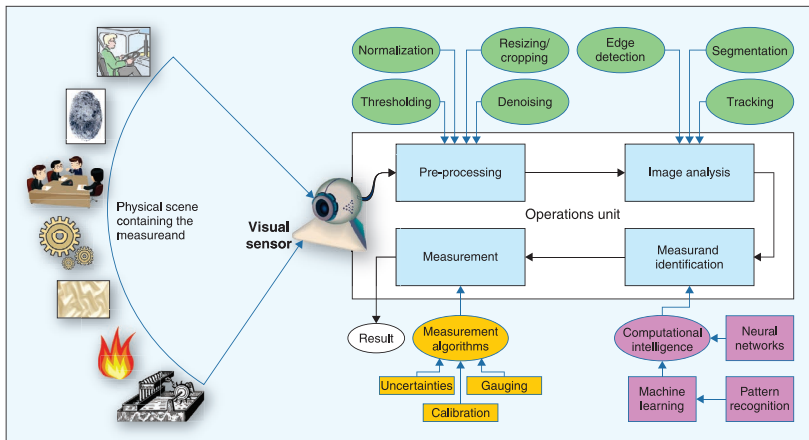


Figura 1: Arquitectura de alto nivel de la medición basada en visión. De izquierda a derecha: la imagen es adquirida por un sensor visual y se envía a la unidad de operaciones para realizar el procesamiento de imágenes (verde), inteligencia computacional (violeta) y operaciones de medición (amarillo).²

²Shirmohammadi, S., Ferrero, A. (2014). Camera as the instrument: The rising trend of vision-based measurement. IEEE Instrumentation Measurement Magazine, 17(3), 41-47.

Deployment de modelos de basados en datos

En el dispositivo (edge / embedded)

- ▶ Ejecutado en hardware local
- ▶ Tiempo real, bajo consumo
- ▶ Sin necesidad de conectividad
- ▶ Ej: Raspberry Pi, Arduino, Jetson

En la nube (cloud/deployment)

- ▶ Escalabilidad alta
- ▶ Requiere conexión estable
- ▶ Alto poder computacional
- ▶ Ej: AWS, GCP, Azure

EDGE vs. CLOUD COMPUTING



- ▶ Un sistema embebido es similar a una computadora convencional.
- ▶ Está compuesto por varios elementos, como el [hardware](#) y el [software](#).
- ▶ Ambos son componentes críticos que deben comprenderse a nivel de sistemas embebidos.

Hardware



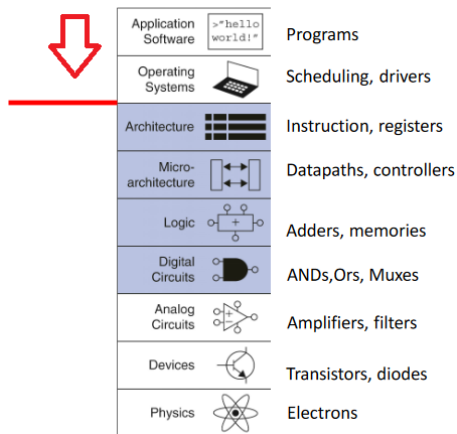
Software

Capas de abstracción



Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

- Dígle a un sistema preconstruido qué hacer (CPU, Many-Cores, GPUs).
- Tenemos que jugar con lo que tenemos aquí.



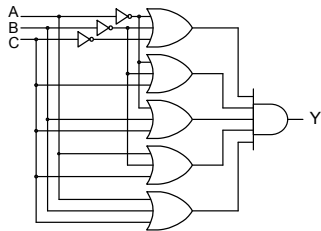
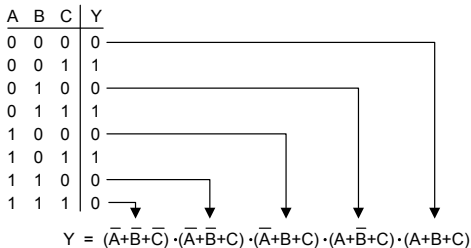
Fundamentos de arquitecturas computacionales

► Especificación

- Tablas de verdad
- Diagramas de estado

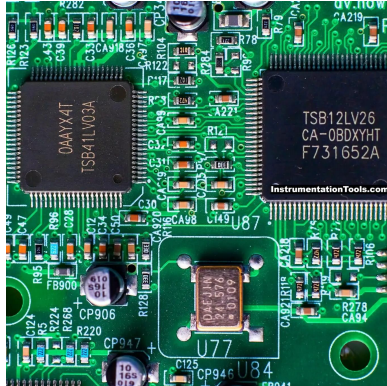
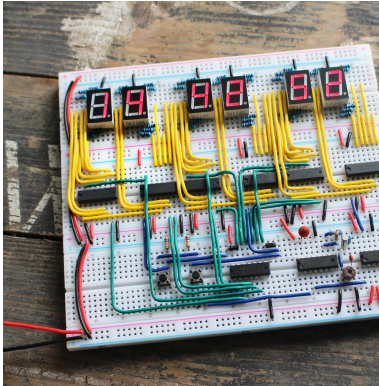
► Diseño

- Optimización local y traducción manual a puertas lógicas
- Captura esquemática

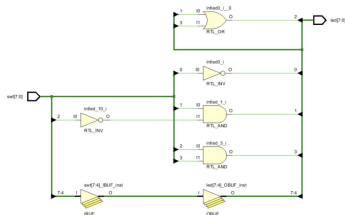


► Implementación

- Mapeo a puertas discretas
- Prototipo, pruebas y depuración (placa de pruebas)
- Implementación final




```
module logic_design (  
    input  logic [7:0] swt,  
    output logic [7:0] led  
);  
  
    // Operaciones lógicas individuales  
    assign led[0] = ~swt[0];  
    assign led[1] = swt[0] & swt[1];  
    assign led[2] = ~swt[2] | swt[1];  
    assign led[3] = swt[2] & swt[3];  
  
    // Asignación directa con buffers a los LEDs 7:4  
    assign led[7:4] = swt[7:4];  
  
endmodule
```



► ¡Los HDL no son lenguajes de programación!

- Estás describiendo un sistema
- No se compila un código HDL
- No se ejecuta un código HDL

► ¡El software no obedece las leyes de la física, el hardware sí!

Hardware para inferencia en ML

El hardware se refiere al tipo específico de dispositivo en el que se ejecuta un modelo de machine learning durante la fase de **inferencia**.

- ▶ La elección del hardware depende de factores como:
 - Requisitos de rendimiento (latencia, throughput).
 - Limitaciones del entorno (consumo energético, tamaño físico).
 - Coste y accesibilidad.
- ▶ Tipos de hardware especializados para inferencia:
 - CPUs (Central Processing Units)
 - GPUs (Graphics Processing Units)
 - FPGAs (Field-Programmable Gate Arrays)
 - Aceleradores específicos como:
 - TPUs (Tensor Processing Units)
 - NPUs (Neural Processing Units)
- ▶ Las **GPUs** son actualmente las más utilizadas en entornos de inferencia gracias a su capacidad de cómputo paralelo masivo y disponibilidad comercial.

Arquitecturas paralelas y granularidad de cómputo

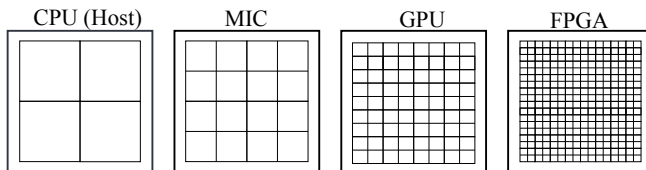


Figura 2: Combinación de CPU y aceleradores especializados permite granularidad mixta.

- ▶ Una **CPU convencional**, combinada con aceleradores, puede distribuir tareas en subprocesos paralelos³.
- ▶ Ejemplos de aceleradores especializados:
 - **MIC (Many Integrated Cores)**⁴.
 - **GPUs** (Graphics Processing Units)
 - **FPGAs** (Field-Programmable Gate Arrays)
- ▶ Estas tecnologías permiten optimizar el rendimiento en tareas de inferencia con alta demanda de procesamiento.
- ▶ Las arquitecturas heterogéneas son comunes en entornos **edge y cloud** donde se busca equilibrio entre eficiencia energética y velocidad de ejecución.

³Granularidad mixta: la capacidad de dividir tareas en múltiples niveles de paralelismo.

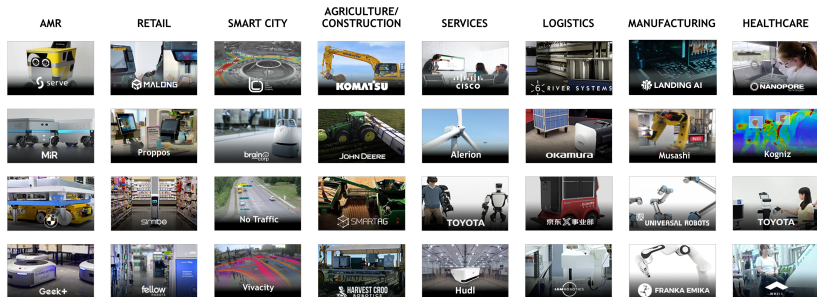
⁴Ej. **Intel Xeon Phi**: procesadores con decenas de núcleos, diseñados para computación paralela intensiva.

Paradigmas de ejecución

- ▶ **CPU:** Optimizada para **latencia baja** y tareas secuenciales complejas. Pocos núcleos potentes, con jerarquía de caché profunda.
- ▶ **GPU (Graphics Processing Unit):** Optimizada para **alto throughput**. Miles de núcleos ligeros en modelo **SIMT** (Single Instruction, Multiple Threads), ideales para cómputo paralelo masivo.

Implicación en IA y CUDA

- ▶ Las CPUs gestionan el *control de flujo* y tareas no paralelizables.
- ▶ Las GPUs ejecutan *kernels CUDA*, **acelerando operaciones vectoriales/-matriciales** (ej. entrenamiento e inferencia en Deep Learning).



- *Safe-critical*
- *Resource-constrained devices*

Toolchains para inferencia eficiente en hardware especializado



Figura 3: Ecosistema de herramientas para despliegue optimizado de modelos de ML.



Figura 3: Ecosistema de herramientas para despliegue optimizado de modelos de ML.

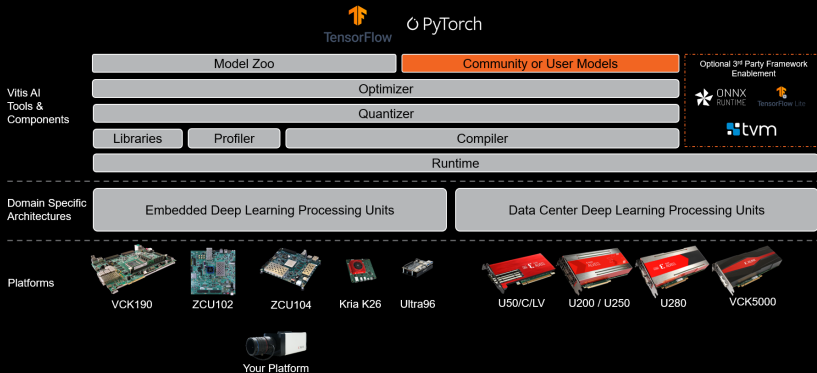
De entrenamiento a inferencia en dispositivos edge o especializados

El despliegue de modelos de *machine learning* en hardware especializado requiere una cadena de herramientas que permita la conversión, optimización y compilación del modelo según el entorno de destino.

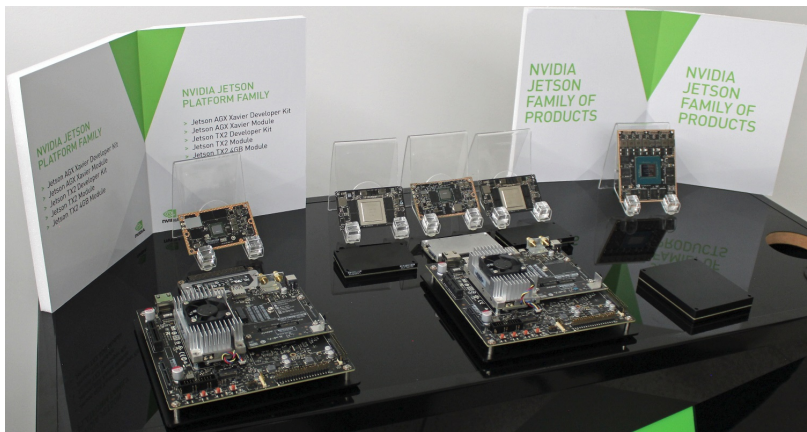
► Entrenamiento y exportación:

- **PyTorch**: framework de entrenamiento flexible.
- **ONNX (Open Neural Network Exchange)**: formato intermedio para facilitar la portabilidad entre frameworks y herramientas de inferencia.

Vitis™ AI Integrated Development Environment



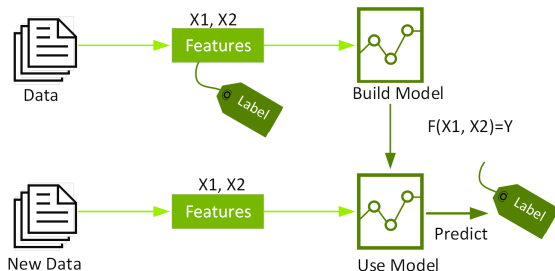
Nvidia Jetson - Características de los dispositivos



Plataforma	Núcleos GPU	Memoria (GB)	CPU
Nvidia GeForce RTX 3060	—	12 (GDDR6) + 36 RAM	Intel i3-12100F
Nvidia GeForce RTX 2060	—	6 (GDDR6) + 12 RAM	Intel i7-10750H
Jetson Xavier NX	384	8 RAM (compartida)	ARMv8.2, 6 núcleos, 1.9 GHz
Jetson Xavier AGX	512	16 RAM (compartida)	ARMv8.2, 8 núcleos, 2.3 GHz
Jetson Orin Nano	1024	8 RAM (compartida)	ARMv8.2, 6 núcleos, 1.7 GHz
Jetson Orin AGX	2048	64 RAM (compartida)	ARMv8.2, 12 núcleos, 2.2 GHz

CUDA no se limita al deep learning. También es una herramienta poderosa para acelerar algoritmos de machine learning clásico.

- ▶ Muchos algoritmos clásicos de ML involucran operaciones intensivas que son altamente paralelizables:
 - Multiplicación de matrices (regresión, PCA).
 - Reducción/suma (cost functions, normalización).
 - Cálculo de distancias (KNN, clustering).
 - Histogramas y búsqueda de divisiones (árboles de decisión, random forest).
- ▶ CUDA permite ejecutar estas operaciones de forma masiva y paralela, acelerando significativamente el entrenamiento y la inferencia.
- ▶ Especialmente útil en conjuntos de datos grandes o entornos con restricciones de tiempo.



► K-Nearest Neighbors (KNN) y K-means

- Comparación en paralelo de vectores y cálculo de distancias Euclidianas.

► Árboles de decisión y Random Forest

- Cálculo paralelo de histogramas y evaluación de splits en nodos.
- Herramientas como **XGBoost** y **LightGBM** usan CUDA para acelerar entrenamiento.

► K-Means y clustering

- Asignación de puntos a centroides y actualización: operaciones paralelas.

¿Por qué usar CUDA en ML clásico?

La CPU ejecuta tareas secuencialmente con pocos núcleos. La GPU (vía CUDA) permite paralelizar tareas sobre miles de núcleos.

- ▶ En tareas como KNN o regresión con millones de muestras, la paralelización reduce el tiempo de cómputo de minutos a segundos.
- ▶ A diferencia del deep learning, donde el cómputo está dominado por entrenamiento con backpropagation, en ML clásico la ventaja de CUDA viene del procesamiento vectorizado de:
 - Distancias entre puntos.
 - Histogramas en árboles.
 - Sumas acumuladas.



- ▶ CUDA (Compute Unified Device Architecture) es la plataforma de cómputo paralelo creada por NVIDIA.
- ▶ Permite a los desarrolladores utilizar la GPU como coprocesador para tareas generales.
- ▶ Basada en el paradigma de Heterogeneous Computing: la CPU (host) gestiona el control, mientras la GPU (device) ejecuta kernels masivamente paralelos.



Relevancia en Inteligencia Artificial

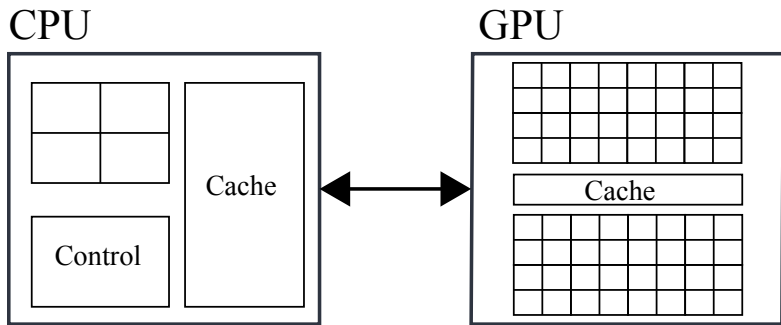
- ▶ Ideal para cómputo intensivo en **Deep Learning**, visión por computador y simulaciones físicas.
- ▶ Compatible con C, C++, Fortran y Python (a través de librerías como PyCUDA y CuPy).
- ▶ Base para frameworks de optimización como **TensorRT**.

Arquitectura Host-Device

- ▶ La **CPU (host)** gestiona el control y la memoria principal.
- ▶ La **GPU (device)** ejecuta *kernels* en miles de hilos en paralelo.

Jerarquía de ejecución

- ▶ Los *kernels* se organizan en una **rejilla (grid)** de **bloques (blocks)**, cada uno con múltiples **hilos (threads)**.
- ▶ Cada hilo se identifica con índices especiales: `threadIdx`, `blockIdx`, `blockDim`, `gridDim`.



Ejemplos de algoritmos clásicos acelerados:

► Regresión Lineal (cuML)

- Multiplicaciones de matrices y reducción paralela.

► Random Forest (cuML / XGBoost)

- Evaluación de splits en paralelo.

► KNN (cuML)

- Comparación masiva de vectores en paralelo.

* Todas estas operaciones se aceleran gracias a CUDA en la GPU.

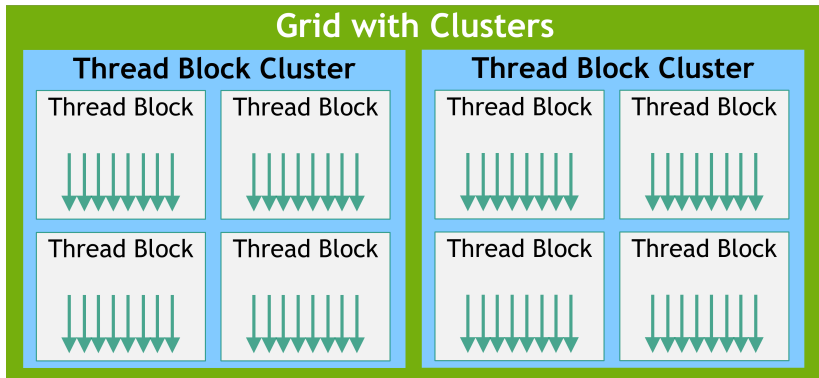
```
1 import cuml
2 from cuml.linear_model import
   LinearRegression
3 from cuml.datasets import make_regression
4 import cupy as cp # Para manejo de
   arrays en GPU
5
6 # Generar datos sintéticos directamente
   en GPU
7 X, y = make_regression(n_samples=10000,
8                       n_features=20,
9                       noise=0.1,
10                      random_state=42)
11
12 # Entrenar modelo de regresión lineal en
   GPU
13 model = LinearRegression()
14 model.fit(X, y)
15
16 # Predecir
17 y_pred = model.predict(X)
```

- ▶ Acelera aplicaciones de deep learning y cómputo científico.
- ▶ Permite optimización mediante paralelismo masivo.

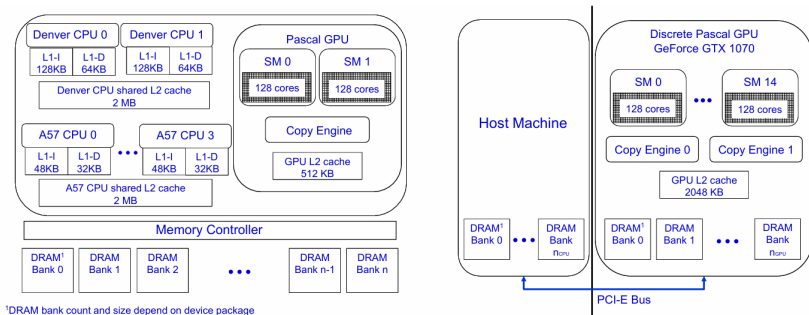
Conceptos Claves en CUDA

- ▶ **Hilos (Threads):** Unidades de ejecución paralela.
- ▶ **Bloques de Hilos (Thread Blocks):** Agrupaciones de hilos que ejecutan tareas conjuntas.
- ▶ **Grid:** Conjunto de bloques de hilos.
- ▶ **Memoria Global y Compartida:** Estrategias de almacenamiento para optimizar el rendimiento.
- ▶ **GPUs Discretas:** Son chips separados de la CPU, conectados mediante buses como PCIe (común en PCs de escritorio).
- ▶ **GPUs Integradas:** Están en el mismo chip que la CPU, compartiendo memoria y recursos.

Grid with Clusters



- ▶ **Hilos (Threads):** Unidades de ejecución paralela.
- ▶ **Bloques de Hilos (Thread Blocks):** Agrupaciones de hilos que ejecutan tareas conjuntas.
- ▶ **Grid:** Conjunto de bloques de hilos.



- Diagrama de comunicación entre *host* y *device*: GPU integrada (izquierda) y GPU discreta (derecha)⁵.
- CUDA es un modelo de programación **heterogéneo** que utiliza tanto la CPU como la GPU.

⁵Ming Yang, et al, "Avoiding Pitfalls when Using NVIDIA GPUs for Real-Time Tasks in Autonomous Systems," *Proceedings of the IEEE Real-Time and Embedded Technology and Applications Symposium*, 2021.

Siguiendo la nomenclatura ampliamente utilizada y aceptada en el contexto de GPUs, nos referiremos al procesador principal como host y al coprocesador como device.

- ▶ Existen distintos elementos clave en el procesamiento paralelo dentro de una GPU:
 - CPU Cores
 - GPU Cores
 - Streaming Multiprocessor

► CPU Cores

- Unidades de procesamiento de una CPU.
- Especializados en ejecutar tareas secuenciales con alta eficiencia.

► GPU Cores

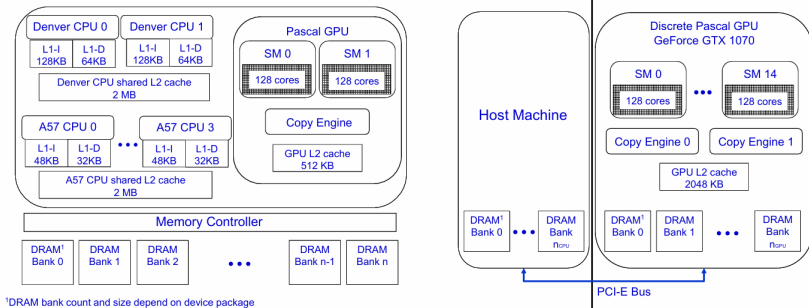
- Unidades de procesamiento dentro de una GPU.
- Optimizados para operaciones en paralelo, ideales para procesamiento masivo de datos⁶.
- No pueden ejecutar sistemas operativos ni gestionar tareas autónomamente.
- Funcionan como coprocesadores bajo instrucciones de la CPU.

► Streaming Multiprocessor (SM)

- Unidad dentro de las GPUs de Nvidia que agrupa múltiples GPU Cores.
- Ejecuta múltiples hilos de procesamiento simultáneamente.
- Coordina la ejecución paralela de un gran número de operaciones.

⁶ como el renderizado de gráficos o algoritmos de machine learning

Elementos del Procesamiento Paralelo en GPUs



► Implementación *system-on-chip* (SoC).

- Esta integra la GPU, los *CPU Cores*, la memoria (*Unified Memory*) y los conectores externos en una pequeña computadora de una sola placa. La GPU integrada comparte recursos de hardware, como la memoria, con los *CPU Cores*.

► GPU discreta

- GPUs discretas, consisten únicamente en los *Streaming Multiprocessors* (SMs) y la memoria local del dispositivo (*device memory*).
- Empaquetados en una tarjeta para su instalación en una ranura de expansión PCIe de la placa base de una computadora.

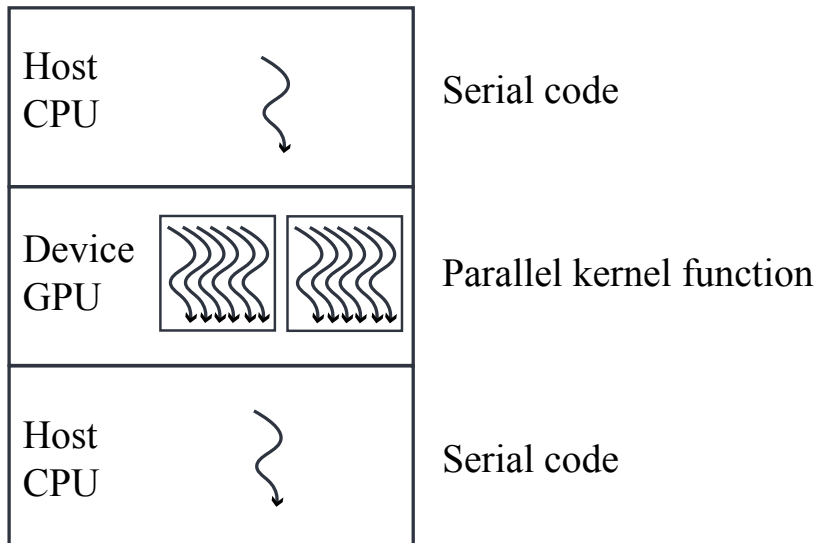
Niveles populares de software y hardware.

Capa	Ejemplos
Aplicación	Visión por computadora Control Automático Reconocimiento de voz Conducción autónoma
Modelo	ResNet - MobileNet - MobileViT YOLO - U-Net
Framework	blue!20 PyTorch - TensorFlow - Keras
Graph Format	blue!20ONNX blue!20NNEF ⁷
Inference Optimizer	TensorRT ARM Compute Library Intel OpenDNN AMD ML Open Xilinx Vitis AI
Hardware	GPU CPU FPGA Aceleradores especializados (TPU, NPU, etc.)

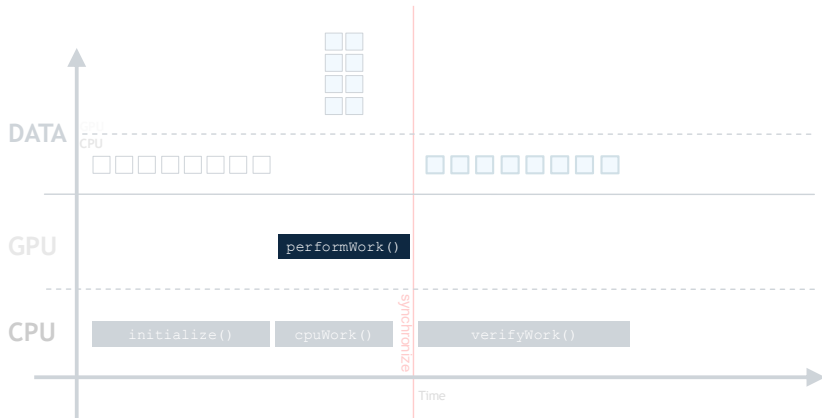
⁷Sintaxis similar a JSON o Python, lo que facilita su comprensión y edición manual

Funcionamiento de CUDA y Kernels

- ▶ Un programa CUDA inicia su ejecución en el **host** (CPU). El **device** (GPU) realiza las tareas de cálculo.
- ▶ **Pasos generales de un programa CUDA:**
 1. Asignación de memoria en la GPU.
 2. Transferencia de datos de la CPU a la GPU.
 3. Ejecución de **kernels** en la GPU.
 4. Copia de resultados de la GPU a la CPU.
 5. Liberación de memoria.
- ▶ **Conceptos importantes:**
 - **Kernels:** Funciones que se ejecutan en la GPU.
 - **Kernel launch:** Proceso de ejecución de un kernel.
- ▶ **Rendimiento:**
 - Cada kernel launch introduce retardos.
 - Optimización:
 - Reducir kernel launches.
 - Seleccionar kernels adecuados.
 - **Inference optimizers** realizan estas optimizaciones.



CUDA Kernel Execution



- ▶ CPU prepara datos y lanza el *kernel*.
- ▶ GPU ejecuta trabajo masivo en paralelo.
- ▶ CPU puede continuar con otras tareas.
- ▶ **Sincronización:** esperar resultados antes de usarlos. `verifyWork()` indica que el host verifica los datos y luego los usa en otro proceso.

- ▶ Estudios sobre GPUs Nvidia y CUDA revelan que aspectos críticos como:
 - Planificación de tareas.
 - Sincronización.
 - Ejecución.
 - Bloqueo de tareas.
- ▶ No están completamente documentados ni son públicamente accesibles.
- ▶ Naturaleza de “caja negra” de la GPU:
 - La falta de documentación convierte el comportamiento interno de la GPU en una “caja negra” para los desarrolladores.
 - Los desarrolladores deben depender de **evidencia empírica** para inferir cómo se comportan los programas CUDA.



- ▶ Libro: *Programming Massively Parallel Processors* de Kirk y Hwu.
- ▶ Tutoriales en línea y cursos especializados en computación paralela.

Microcontroladores

Microcontrolador vs Microprocesador

Microprocesador:

- ▶ Parte de un sistema más grande.
- ▶ Requiere componentes **externos** para memoria y almacenamiento.
- ▶ Usado en computadoras y sistemas de propósito general.
- ▶ Se pueden encontrar en laptops, servidores y computadores.
- ▶ Ofrece flexibilidad en el diseño.

Microcontrolador:

- ▶ Sistema integrado.
- ▶ Memoria, almacenamiento y procesador **internos** en un solo chip.
- ▶ Usado en dispositivos de tareas específicas o **computación especializada**.
- ▶ Telefonía celular, reproductores MP3, entre otros.
- ▶ Limitada flexibilidad en el diseño.

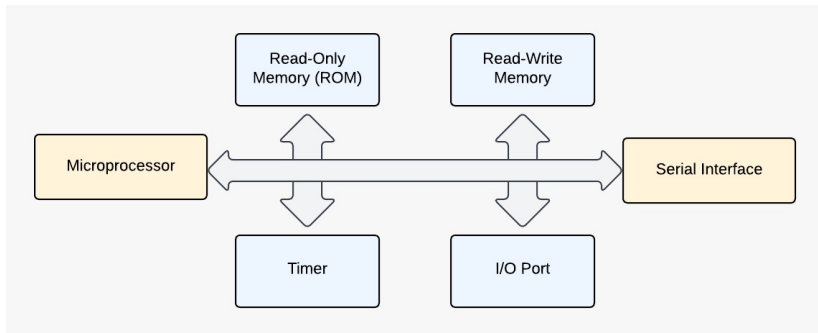
Diferencias entre microcontroladores y microprocesadores

	Microprocessor	Microcontroller
Platform		
Compute	1GHz–4GHz	1MHz–400MHz
Memory	512MB–64GB	2KB–512KB
Storage	64GB–4TB	32KB–2MB
Power	30W–100W	150μW–23.5mW

Gran diferencia en consumo energético.

Características de los Microcontroladores

El microcontrolador es una parte de un rompecabezas



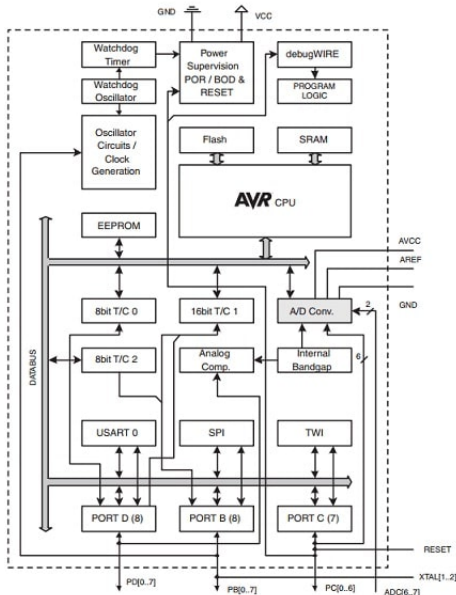
- **Velocidad de Reloj:** Entre 1 Hz y 4 MHz.
- **Memoria:** Generalmente de 2 KB a 512 KB.
- **Almacenamiento:** Desde unos pocos kilobytes hasta megabytes.
- **Consumo de Energía:** Muy bajo, en el rango de microwatts o milliwatts.
- **Tamaño:** Compacto, ideal para dispositivos pequeños.

Aplicaciones comunes de microcontroladores

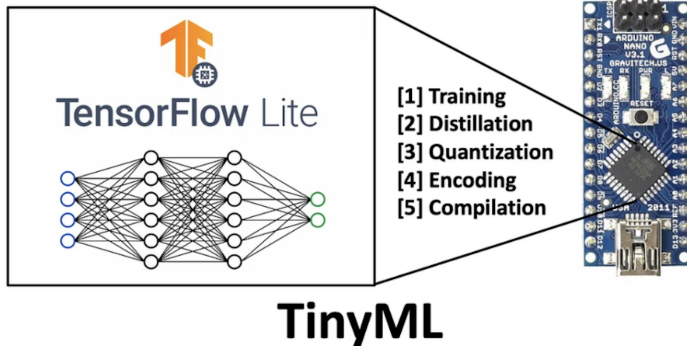


- ▶ Sistemas de control automatizado (por ejemplo, control de motores).
- ▶ Dispositivos médicos portátiles (por ejemplo, monitores de ritmo cardíaco).
- ▶ Electrodomésticos inteligentes (por ejemplo, lavadoras, microondas).
- ▶ Sistemas de IoT (Internet of things).
- ▶ Juguetes electrónicos y dispositivos portátiles.

Arquitectura básica de un microcontrolador ATmega328p

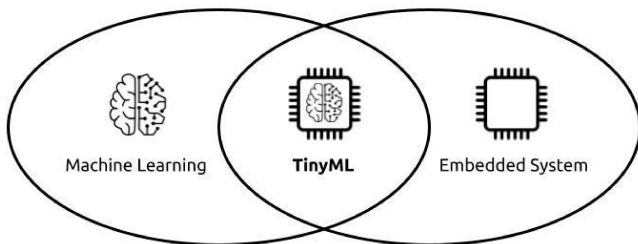
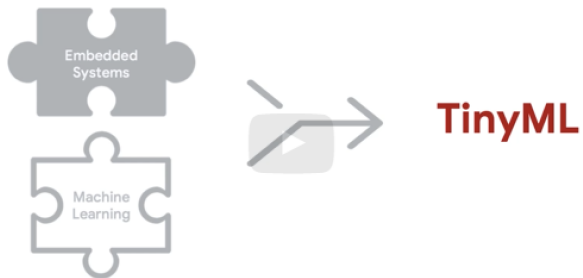


- El **Watchdog Timer** es un temporizador interno en los microcontroladores diseñado para detectar y recuperar el sistema en caso de fallas.
- El **Bandgap Interno** es una referencia de voltaje estable. Calibración interna del ADC.
- El **TWI**, también conocido como I2C (Inter-Integrated Circuit), es un protocolo de comunicación serial que utiliza dos líneas.
- Módulos de comunicación como UART, SPI, e I2C.



TinyML es un paradigma centrado en la implementación de modelos de *deep learning* en dispositivos embebidos con recursos computacionales limitados, principalmente en microcontroladores basados en CPUs de propósito general de bajo costo.

En este contexto, se han desarrollado frameworks especializados como Tensorflow Lite y Edge Impulse.



Hardware:

- ▶ Computo: Procesadores con frecuencia limitada (1 Hz - 4 Hz).
- ▶ Memoria: Tamaños reducidos (2 KB - 512 KB).
- ▶ Almacenamiento: Kilobytes a megabytes.

Consumo energético:

- ▶ Bajo consumo (microwatts a milivatios).
- ▶ Ideal para tareas con tiempo de ejecución limitado.

Desafíos prácticos:

- ▶ Adaptación de algoritmos de aprendizaje automático a recursos limitados.
- ▶ Optimización para tareas específicas en tiempo real.
- ▶ Selección adecuada del microcontrolador según la tarea.
- ▶ Garantizar rendimiento bajo restricciones severas de hardware.
- ▶ Integración eficiente de sistemas embebidos y modelos de aprendizaje automático.

- ▶ Los modelos de Machine Learning (ML) se adaptan a estos recursos de un MCU utilizando técnicas como:
 - **Reducción de modelo:** Uso de modelos más pequeños (ej. redes neuronales ligeras).
 - **Optimización:** Conversión de modelos entrenados a formatos optimizados (ej. TensorFlow Lite).
 - **Algoritmos de ML eficientes:** Como Árboles de Decisión, K-NN, o SVM para clasificación simple.
- ▶ **Pasos para la implementación:**
 1. **Entrenamiento del modelo** en una plataforma potente (PC o servidor).
 2. **Conversión del modelo** a un formato adecuado para el microcontrolador (por ejemplo, TensorFlow Lite).
 3. **Desarrollo en el microcontrolador:** Integración del modelo en el código embebido y optimización de recursos.
 4. **Inferencia:** El microcontrolador ejecuta el modelo para hacer predicciones en tiempo real.

¿Cómo desplegar árboles de decisión en microcontroladores?

► 1. Conversión manual a código C/C++

- Para árboles muy simples, puedes convertir el modelo a una estructura if-else.

```
if (feature_1 <= 2.5) {  
    if (feature_2 <= 1.7) return 0;  
    else return 1;  
} else {  
    return 1;  
}
```

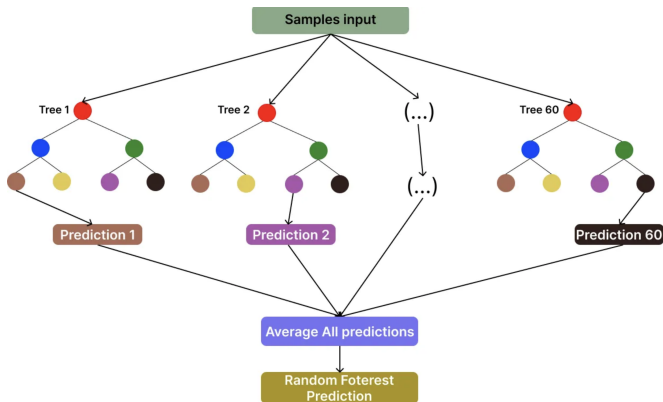
► 2. Usar librerías de conversión automática

- **micromlgen** convierte modelos de scikit-learn en código C listo para microcontroladores.
- Compatible con árboles de decisión, random forests pequeños, etc.
- **EloquentTinyML**⁸ permite integrar fácilmente modelos ligeros en placas como Arduino Nano 33 BLE Sense.

Ideal para aplicaciones de clasificación en dispositivos embebidos con recursos limitados.

⁸https://github.com.translate.google/eloquentarduino/EloquentTinyML?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

Exportar modelos a microcontroladores con micromlgen



```
!pip install micromlgen
```

La librería `micromlgen` permite convertir modelos simples de `scikit-learn` (como árboles de decisión o pequeños Random Forests) en código C compatible con microcontroladores. Ideal para integrar modelos de ML clásicos en plataformas embebidas como Arduino, ESP32 o STM32.

Exportación de Random Forest a C++ con micromlgen

- ▶ **micromlgen** permite exportar modelos de `scikit-learn` a código C++ embebible.
- ▶ Ideal para correr modelos ligeros en microcontroladores (Arduino, ESP32, STM32).
- ▶ El modelo Random Forest se traduce como:
 - Cada árbol se convierte en una estructura `if-else`.
 - Cada árbol vota por una clase.
 - Se elige la clase con más votos.

Ejemplo de código generado:⁹

```
if (x[3] <= 0.8) {  
    votes[0] += 1;  
} else {  
    if (x[2] <= 5.05) {  
        votes[1] += 1;  
    } else {  
        votes[2] += 1;  
    }  
}  
  
// Repetido para los 100 árboles
```

⁹<https://medium.com/@thommaskevin/tinymml-random-forest-classifier-and-regressor-b351aa0980e8>



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Deployment

EIN087B - Ciencia de Datos
Paralelo 701

Profesor: Jorge Portilla G.
`jorge.portilla@usm.cl`

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile